

Backstop: Recovery-Safe GPU Sharing Between Neural Receivers and AI Inference in AI-RAN

ANONYMOUS AUTHOR(S)

As AI-RAN platforms share GPUs between radio processing and AI inference, neural receivers introduce work that GPU schedulers do not track. Until its neural receiver (NeuralRx) completes, an uplink transport block (TB) has a pending recovery: a conventional decoding that is absent from the GPU queue but mandatory if the receiver fails. This paper presents Backstop, a multi-GPU scheduler that tracks these pending recoveries explicitly. Backstop admits speculative receivers, retimes recovery, and grants GPU time to external AI only when an independent verifier confirms that every unresolved TB can still recover if all of them fail. Backstop keeps each accepted decision valid through CUDA launch, inter-GPU transport, and physical completion with bounded revalidation, latest-start enforcement, and freeing of GPU time only after confirmed completion. On two four-GPU A100 nodes under a qualified synthetic timing contract, Backstop records 0/120 safety violations in physical boundary diagnostics, where idle-time admission records 40/40. Across 13,716 feasible capacity-model points, Backstop admits more AI capacity than static reservation at 26.2%, and its p99 decision time is 1.494 ms at 64 pending recoveries.

CCS Concepts: • **Computer systems organization** → **Distributed architectures**; **Real-time systems**; • **Networks** → *Network resources allocation*.

Additional Key Words and Phrases: AI-RAN, GPU scheduling, neural receiver, conditional recovery, MPS, admission control

ACM Reference Format:

Anonymous Author(s). 2027. Backstop: Recovery-Safe GPU Sharing Between Neural Receivers and AI Inference in AI-RAN. In *Proceedings of ACM SIGMETRICS 2027 (SIGMETRICS '27)*. ACM, New York, NY, USA, 26 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

As radio access networks (RANs) move baseband processing onto GPUs, AI-RAN platforms run radio physical-layer (PHY) processing and AI inference on the same accelerators [3, 14, 19, 22]. To use the GPU capacity that radio processing leaves idle, these platforms admit AI services, such as large language model (LLM) inference, alongside deadline-critical PHY tasks [12, 21]. This co-location is effective only when every radio task still completes before its processing deadline [7], and conventional GPU sharing can disrupt 5G processing [19]. Neural receivers make this requirement harder to enforce. A neural receiver replaces channel estimation, equalization, and demapping with a learned model whose decoding outcome is known only after it executes [6, 10]. However, the GPU schedulers of these platforms protect only the radio work that is already visible in a GPU queue or in a resource allocation.

The scheduling challenge stems from the fallback structure of neural reception. A neural receiver (NeuralRx) is optional work: a timely success delivers the uplink transport block (TB), while a failure, a late result, or a lost result makes the conventional receiver mandatory for the same TB. Until the NeuralRx outcome resolves, this conventional recovery is absent from the GPU queue, but it must remain executable before the radio deadline. When several cells run NeuralRx together, their recoveries can contend for one conventional decoder, and their failures can be correlated. In addition, a Multi-Process Service (MPS) share limits the threads of a co-running process, but it does not bound host scheduling, queueing, or process lifecycle [15]. Thus, a scheduler that admits AI into apparently idle GPU time can push a future recovery past its deadline, and a scheduler that reserves every possible recovery leaves the capacity released by NeuralRx success unused.

Table 1. Closest-work distinction in public designs.

System	Primary protected/allocated object	State not exposed to its scheduler
YinYangRAN, CloudRIC, Concordia	Fraction, queue/deadline, or reserved capacity	Unqueued same-TB pending recovery
DARIS, REEF, XSched, nvtaskset	GPU stage, kernel, context, or partition	Radio recovery, transport ownership, and single commit in one verified schedule
ARCHES, OCUDU, SMEC	PHY choice, timing class, or edge SLO	Cross-instance all-fail recovery set plus atomic external-AI slot
Backstop	Same-TB pending recovery	Explicitly modeled, verified, and carried through physical completion

To share accelerators between radio processing and other workloads, existing systems employ different approaches. As summarized in Table 1, these systems differ in the object that their schedulers protect or allocate. vRAN and AI-RAN resource managers [3, 7, 12, 19–22] allocate compute fractions, queued requests, reserved capacity, or worker placement between RAN functions and other workloads. GPU scheduling systems [4, 5, 9, 23] partition, prioritize, or preempt GPU stages, kernels, and contexts. PHY and edge platforms [16, 18, 25] select PHY models, expose timing classes inside the distributed unit (DU), or coordinate edge service-level objectives (SLOs). However, their scheduling units are resources or queued jobs. Their public designs neither represent the recovery that a speculative receiver may create nor keep an admission decision valid through GPU launch and completion, where a decision that is valid when made can become invalid before it executes.

In this paper, we present Backstop, a recovery-safe multi-GPU scheduler that shares AI-RAN GPUs between neural receivers and AI inference. Backstop tracks each unresolved recovery as a *pending recovery* and accepts a scheduling decision only when an independent verifier confirms that every unresolved TB can still recover if all of them fail. Specifically, Backstop introduces 1) a recovery-aware scheduling model that jointly decides NeuralRx admission, recovery timing, and external-AI slots over one set of pending recoveries shared by several cuPHY instances, 2) schedule enforcement that keeps each accepted decision valid through CUDA launch, IPC/P2P transport, and physical completion with bounded revalidation, absolute latest-start enforcement, atomic outcome batches, and freeing of GPU time only after confirmed completion, and 3) an admission limit, qualified by the measurement conditions of every bound, that classifies each state as SAFE+AI, SAFE-ONLY, INFEASIBLE, or UNQUALIFIED before execution. Our evaluation on two four-GPU A100 nodes under a qualified synthetic timing contract shows that physical boundary diagnostics record 0/120 safety violations for Backstop and 40/40 for idle-time admission. Under bursty BurstGPT-based arrivals with a 200 ms AI SLO, Backstop serves 6.2-18.5% more on-time AI tokens than its recovery-first ablation and static reservation serves none, while idle-time admission misses the recovery deadline in up to 10.0% of periods. In 1,200 physical release periods, the shared lane finishes every period before its scheduled end, and under seven injected fault classes every TB commits exactly once before expiry. Across 13,716 feasible capacity-model points, Backstop admits more AI capacity than static reservation at 3,600 (26.2%). The scheduler matches an exact checker on 16,023 states, and its p99 decision time is 1.494 ms at 64 pending recoveries. In the original trace mode, Backstop and recovery-first deliver the same 349,387 timely tokens.

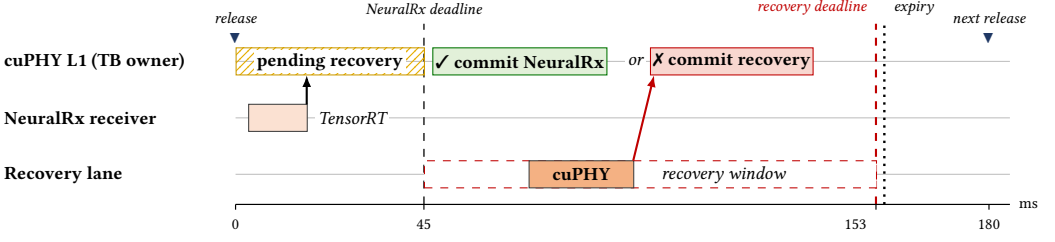


Fig. 1. One uplink TB in one 180 ms release period. Until the 45 ms NeuralRx deadline, the TB has a pending conventional recovery that no GPU queue contains. A failed, late, or missing NeuralRx outcome requires one cuPHY recovery that ends before the recovery deadline.

2 Background and Motivation

2.1 AI-RAN on shared GPUs

AI-RAN platforms execute RAN functions and AI services on the same GPUs [12, 14, 19–22]. In our setting, these GPUs serve three classes of work: optional neural reception, mandatory conventional reception, and external AI.

Neural Reception. An uplink TB is decoded by a conventional receiver, such as the cuPHY physical uplink shared channel (PUSCH) pipeline [14], or by a neural receiver [6, 10]. A cuPHY instance runs the PUSCH pipelines of several cells, and in our evaluated contract each cell sends one TB per release period. In this contract (Figure 1), a TB commits its NeuralRx result when the result passes its cyclic redundancy check (CRC) by the NeuralRx deadline; otherwise, one conventional recovery must complete before the recovery deadline. Figure 2a shows that this outcome is uncertain. Across a synthetic block-Rayleigh channel with 500 TBs per point and the CDL-D and CDL-E models [1, 11] with 50 TBs per point, the NeuralRx success rate rises from near 0% to near 100% within 1 dB of the point at which the conventional receiver decodes half of the TBs. In CDL-D and CDL-E, the conventional receiver also decodes TBs that NeuralRx misses. The channel quality indicator that drives link adaptation targets a 10% transport block error probability [2], which keeps TBs near this transition.

GPU Sharing. MPS allows CUDA processes to execute concurrently and limits the active-thread percentage of each client [15], but it does not bound host scheduling, queueing, context lifecycle, or allocation. Multi-Instance GPU (MIG) partitions provide spatial isolation, but a fixed partition cannot reclaim the capacity that an NeuralRx success releases. Figure 3a co-runs Qwen2.5-1.5B prefill [17] with a two-cell 273-PRB PUSCH pipeline and sweeps the MPS share of Qwen. A single 16-token request keeps the radio p99 latency within 11.9 ms at every share, against 8.9–10.0 ms alone. Eight 512-token requests keep it within 13.9 ms up to an 80% share but reach 21.2–22.9 ms at 100%, where 93 of 1,000 slots miss the 21 ms deadline. Thus, an MPS share is safe only for the AI class under which it is qualified.

Execution Tails. Figure 3b normalizes each latency by its median. The NeuralRx TensorRT graph stays within 3.5% of its median, while end-to-end paths do not. At the 4.5 ms indication deadline of the Aerial testMAC [14], cross-GPU receiver pairs reach 4.0 and 3.7 times their median in a Python path and a partly native C++/CUDA path. Host construction of a schedule reaches 29 times its median. Thus, a schedule computed from kernel execution times can become invalid before it executes.

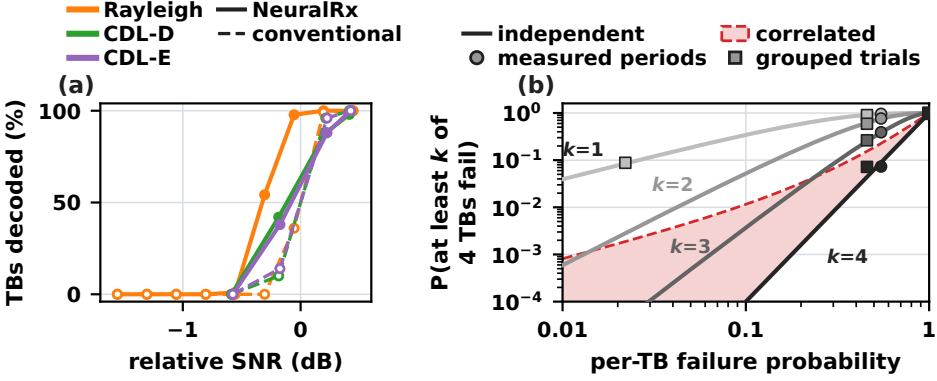


Fig. 2. Uncertain recovery need across channels and failure rates. (a) Share of TBs that NeuralRx (solid) and the conventional receiver (dashed) decode in three channel models; the relative SNR is 0 dB where the conventional receiver decodes half of the TBs. (b) Probability that a period with four admitted TBs requires at least k recoveries when each TB fails with probability p . Lines assume independent failures; the band raises the correlation of the all-four event to 0.3. Circles are measured periods that join the transition-SNR TBs of two consecutive campaign periods; squares are grouped trials of four consecutive receiver trials.

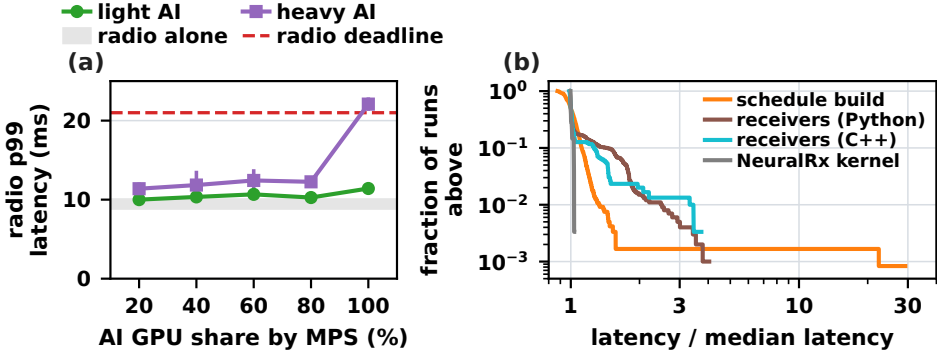


Fig. 3. Execution variability on shared A100 GPUs. (a) Radio p99 of a two-cell 273-PRB PUSCH pipeline against the MPS share of a co-running Qwen prefill class: one 16-token request (light) or eight 512-token requests (heavy). Lines connect run means, and bars span the runs. (b) Fraction of runs whose latency exceeds a multiple of its median: schedule construction on the host (1,200 periods), cross-GPU receiver pairs at the 4.5 ms scale on a Python and a partly native C++/CUDA path (1,000 and 300 radio slots), and the NeuralRx kernel as a TensorRT graph (300 slots).

These properties couple radio correctness to GPU admission decisions that precede the receiver outcome. While existing systems allocate GPU resources between RAN and AI workloads [12, 19–22] or preempt GPU kernels [4, 9, 23], their public designs do not schedule the conditional recovery that neural reception creates.

2.2 Challenges in conditional recovery scheduling

Existing schedulers admit AI against the work in GPU queues or against a fixed reservation [7, 19, 20]. For neural reception, this admission exposes three challenges: uncertain recovery need, one recovery

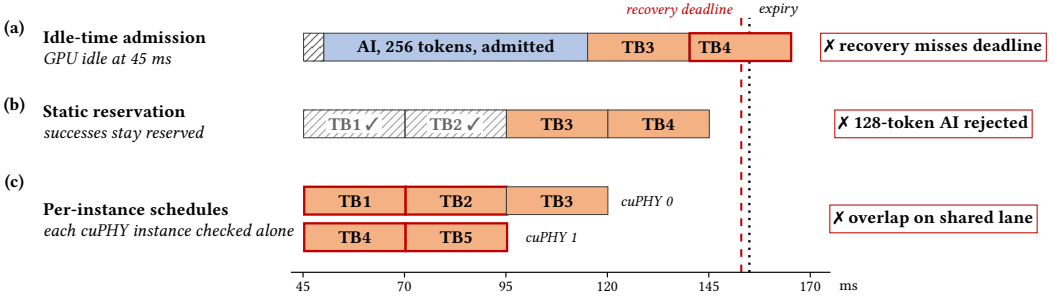


Fig. 4. Scheduling failures after two timely successes (TB1, TB2). (a) Idle-time admission pushes a required recovery past the recovery deadline. (b) Static reservation rejects a 128-token AI request that fits once the successful reservations are released. (c) Per-instance schedules claim the same lane interval.

lane shared by the cells of several cuPHY instances, and decisions that must remain valid through execution.

Pending Recovery. Before the NeuralRx outcome resolves, the conventional recovery of a TB is conditional in the observed execution and mandatory in the failure branch. We call it a *pending recovery*. Figure 2b converts a per-TB failure probability p into the number of recoveries that a release period with four admitted TBs requires; measured periods follow the curves for independent failures. Idle-time admission reserves no recovery and is wrong whenever one recovery is required, which occurs in 34% of periods at $p = 0.1$. Static reservation holds four recoveries, which a period uses only when all four TBs fail, once in 10^4 periods under independent failures. A within-period failure correlation of 0.1 or 0.3 makes this all-four event 16 or 116 times more likely. Thus, planning for the typical number of recoveries is unsafe, and holding every reservation until all outcomes resolve leaves released capacity unused.

Scheduling Failures. Figure 4 shows three scheduling failures after two timely successes in our evaluated contract. Idle-time admission sees an empty queue at the 45 ms NeuralRx deadline, admits a 256-token AI request, and pushes a required recovery past the recovery deadline. Static reservation keeps the reservations of the two successful TBs and rejects a 128-token AI request that fits after they are released. Per-instance schedules are each valid for one cuPHY instance, but they claim the same interval on the shared recovery lane. Thus, a scheduler must represent unresolved recovery explicitly, recompute the schedule at each outcome event, and verify one schedule for the global set of pending recoveries on the contended lane.

Execution Gaps. A feasible schedule must remain feasible when it executes. Table 2 lists seven counterexamples that define this requirement. In our development runs, a 23.5 ms host stall launched a 128-token AI request outside its scheduled interval, and applying simultaneous outcomes in sequence created an intermediate state that never existed. Thus, a scheduler must charge launch control in every schedule, apply simultaneous outcomes atomically, and bind ownership to physical completion. This motivates how Backstop enforces its schedules in Section 3.

Table 2. Counterexamples define what schedule enforcement must cover.

Counterexample	Broken assumption	Required enforcement
Local-safe/global-unsafe	Local schedules compose on a shared lane	One schedule for the global pending-recovery set
Dispatch tail	A verified schedule stays valid until launch	Bounded revalidation and physical latest-start
Sequential outcome replay	Simultaneous outcomes may apply in any order	Atomic outcome batch
Fixed AI slot	Execution time covers launch control	Launch budget and AI busy interval
Reply loss after effect	Retrying ambiguity is harmless	Version invalidation and no AI admission without evidence
Two outstanding prepares	RPC at-most-once implies visible ownership	At most one pending launch ticket
Warm-bound reuse	Lifecycle does not affect service tails	Qualification per measurement condition

3 Backstop Design

Backstop schedules pending recovery as explicit state and keeps each scheduling decision valid until its GPU effect completes. This section presents the architecture, pending-recovery scheduling, schedule enforcement on the GPU, and the event algorithm, and then states the guarantee scope and the implementation.

3.1 Overall architecture

Backstop schedules pending recovery and external AI on the GPUs of one AI-RAN node. Figure 5 shows the architecture of Backstop, which separates a control layer on the host from an execution layer of persistent MPS workers, and follows one release period through seven steps (❶-❷). The figure uses the evaluated example in which TB1 and TB2 succeed, TB3 fails its CRC, TB4 is late, and TB5 is rejected at admission.

Control Layer. One host coordinator holds four control objects. *Decision Pages* record, for each TB, the NeuralRx outcome, the resulting action, and the version of the decision. *Pending Recoveries* lists every TB whose conventional recovery is still required, with its cuPHY instance, its recovery window, and its recovery bound c ; in the example, TB3 and TB4 remain with the 45-153 ms window and $c = 25$ ms, while the entries of TB1 and TB2 are cancelled. *Recovery-Lane Schedule* holds the current schedule X_t of the shared recovery lane and a candidate X' that adds one AI slot with its 5 ms launch budget before the recovery deadline. *Independent Verifier* commits X' or keeps X_t .

Execution Layer. Persistent MPS workers execute all GPU work. In the multi-GPU placement (Figure 5b), GPU0 and GPU1 run the cuPHY instances that own TB1-TB3 and TB4-TB5 at a 50% MPS share. GPU3 runs the TensorRT FP16 NeuralRx worker as a CUDA graph at 80%, receives IQ samples and channel estimates over NVLink P2P, and returns log-likelihood ratios (LLRs). GPU2 runs the shared cuPHY recovery lane at 80% and the Qwen worker at 20% behind a latest-start gate, and it writes the completion markers. In the single-GPU placement (Figure 5a), the same processes share one device and exchange IQ, LLR, and TB buffers through CUDA IPC rings.

Scheduling Loop. Each cuPHY instance publishes the NeuralRx outcome of its TBs to a decision page (❶). At the NeuralRx deadline, one outcome batch updates the pending-recovery table (❷). The candidate scheduler retimes the required recoveries around a candidate AI slot (❸), and the

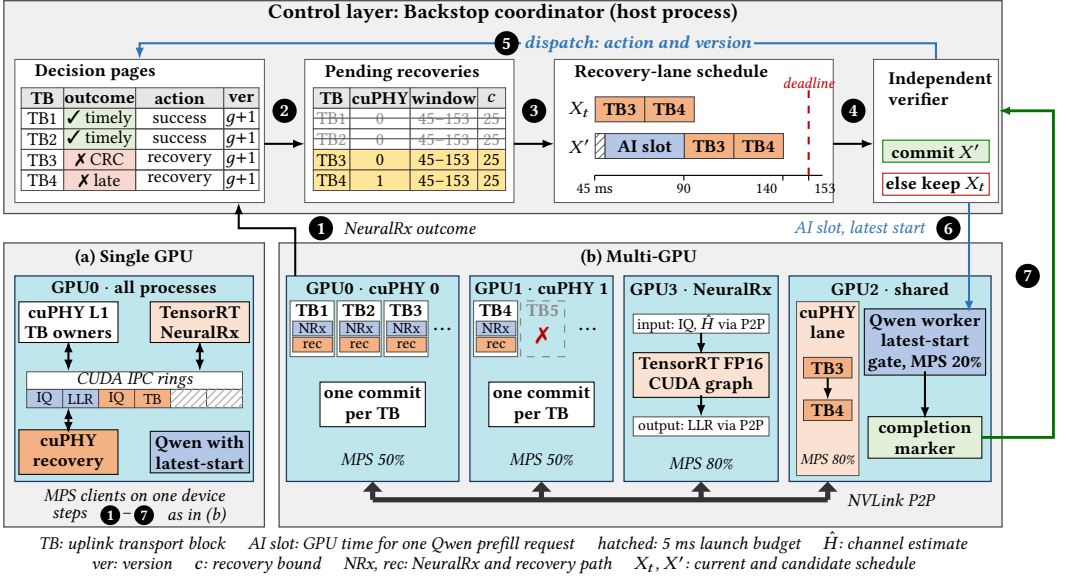


Fig. 5. Backstop architecture for one release period. Steps: NeuralRx outcome (①), outcome batch (②), recovery retiming (③), independent verification (④), dispatch (⑤), AI slot (⑥), and completion marker (⑦). The steps are drawn for the evaluated multi-GPU placement (b); the single-GPU placement (a) exchanges IQ, LLR, and TB buffers through CUDA IPC rings and uses the same steps. In (b), each TB comes from a different cell; cuPHY instance 0 owns TB1-TB3, instance 1 owns TB4 and TB5, and ellipses mark further TBs.

independent verifier checks the all-fail schedule, credits, and version before it commits X' or keeps X_t (④). The coordinator then publishes each action with its version (⑤) and sends the AI slot with an absolute latest-start to the Qwen worker (⑥). A completion or non-launch marker frees the slot (⑦), and the shared lane runs the required recoveries in the committed order.

Scheduling Objects. The scheduler uses only the information that is visible at the current event. A radio job and an external-AI job are

$$J_i = (r_i, d_i, h_i, c_i, K_i, q_i), \quad A_a = (u_a, v_a, \ell_a, b_a, g_a), \quad (1)$$

where r_i and d_i are the release and expiry, h_i is the owning cuPHY instance, c_i is the conventional host-to-commit bound, K_i contains the eligible NeuralRx endpoints, and q_i is the optional radio value; u_a, v_a, ℓ_a, b_a , and g_a are the AI arrival, timely value, deadline, physical-completion bound, and recovery margin. Decisions cannot use future CRC outcomes, future arrivals, realized service times, or hidden channel labels.

Qualified Modes. A service bound is valid for scheduling only under matching measurement conditions

$$\pi = (\text{GPU, placement, MPS, lifecycle, co-run, software}). \quad (2)$$

Therefore, a warm local measurement cannot qualify a cold, restarted, or remote path. The scheduler marks a state UNQUALIFIED whenever a required bound was not measured under the active conditions, even when a sample finishes before its deadline.

Separation of Proposal and Authority. The candidate scheduler may use a fast list heuristic and may reject a feasible placement. It cannot create an unsafe execution because only the verifier can

commit a candidate. With this separation, the evaluation compares decision quality with an exact oracle while safety remains independent of the heuristic.

3.2 Pending-recovery scheduling

An unresolved NeuralRx creates required future work even when no conventional kernel is queued. Backstop records this work as a pending recovery

$$R_i = (a_i(t), d_i - g_i, c_i, h_i), \quad (3)$$

where $a_i(t)$ is its earliest legal start. For the unresolved set $U(t)$, an all-fail schedule assigns every R_i a non-overlapping interval $[s_i, s_i + c_i]$ on its physical recovery lane, after release and before $d_i - g_i$. Contending transport and AI busy intervals belong to the same schedule.

All-Fail Dominance. Backstop needs one schedule, not one schedule per combination of outcomes.

PROPOSITION 1 (ALL-FAIL DOMINANCE). *If a timely NeuralRx success only deletes its conventional recovery and does not increase any remaining job's demand, a feasible all-fail schedule for $U(t)$ is feasible for every failure subset $F \subseteq U(t)$.*

PROOF. Delete successful requests from the all-fail schedule. Release, expiry, capacity, and non-overlap constraints of every remaining interval remain unchanged. $\square$

The reduction replaces $2^{|U|}$ outcome schedules with one executable schedule. It does not apply when a success creates new mandatory work or changes another service bound; that case requires a new mode.

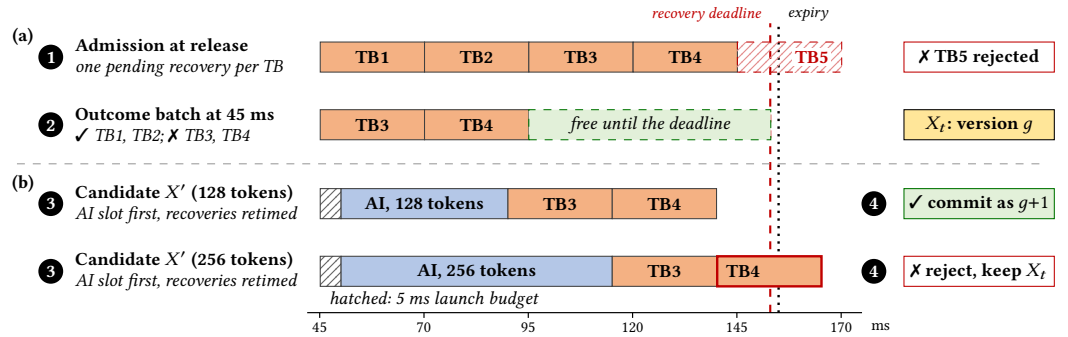


Fig. 6. One transaction on the shared lane. (a) Admission reserves one all-fail recovery per TB and rejects TB5; the 45 ms batch cancels two pending recoveries in one atomic update. (b) The verifier commits the 128-token AI slot and keeps X_t for the 256-token AI slot, whose last recovery would end after the recovery deadline. AI requests are Qwen prefill requests; hatching marks the 5 ms launch budget.

Admission and Outcome Batch. Figure 6a shows one transaction on the shared lane. At release, admission reserves one pending recovery per TB before any optional dispatch (1). TB1-TB4 fit before the recovery deadline, and TB5 is rejected because its reservation would cross the recovery deadline. Therefore, endpoint rejection never removes the conventional path. At the 45 ms NeuralRx deadline, one outcome batch applies all observed outcomes as one atomic update (2): TB1 and TB2 succeed and cancel their pending recoveries, while TB3 and TB4 fail and keep theirs. The capacity that TB1 and TB2 release stays free until the recovery deadline, and the committed schedule X_t

holds two recoveries. Sequential replay of the same outcomes is invalid because it can create intermediate states that never coexisted.

Candidate AI Slot. Figure 6b adds one AI slot to X_t . For a 128-token request, the candidate X' places the 5 ms launch budget and the AI slot first and retimes TB3 and TB4 behind them (⑤). The last recovery still ends before the recovery deadline, and the verifier commits X' as version $g + 1$ (④). For a 256-token request, TB4 would end after the recovery deadline; the verifier rejects X' and keeps X_t .

Global Atomic State. Schedule scope follows the contended lane: disjoint lanes compose by union, while cuPHY instances that share a recovery lane require one global set of pending recoveries. The complete runtime state is

$$X_t = (S_t, Q_t, T_t, L_t, E_t, C_t, G_t), \quad (4)$$

where S is the recovery schedule, Q and T are the endpoint and transport credits, L holds the AI slots, E holds the completion events and worker epochs, C is the radio commit state, and G is the multi-instance AI ownership. The verifier commits a private X' only when

$$\begin{aligned} \text{VALID}(X') \equiv & \text{ExecutableRecovery}(S') \wedge \text{UniqueCredits}(Q', T') \\ & \wedge \text{TimelyAI}(L', E') \wedge \text{CurrentVersions}(X'). \end{aligned} \quad (5)$$

A rejected candidate leaves X_t intact, which eliminates partial calendar updates and orphan AI slots.

Safety Invariant. The verifier and the enforcement in Section 3.3 maintain one invariant across all events.

SAFETY INVARIANT 1 (RECOVERY, OWNERSHIP, AND SINGLE COMMIT). *For every admitted radio request, every allowed unresolved-outcome branch retains a collision-free conventional path completing by expiry. No endpoint, transport, AI, or output credit is reused before matching terminal physical evidence, and each request commits at most one radio result.*

3.3 Enforcing the schedule on the GPU

A feasible host calendar is insufficient unless launch and ownership follow it. Figure 7 follows one committed AI slot L across the coordinator, the marker files, the Qwen worker, and the GPU2 CUDA stream, together with four injected faults and the radio state of period 17.

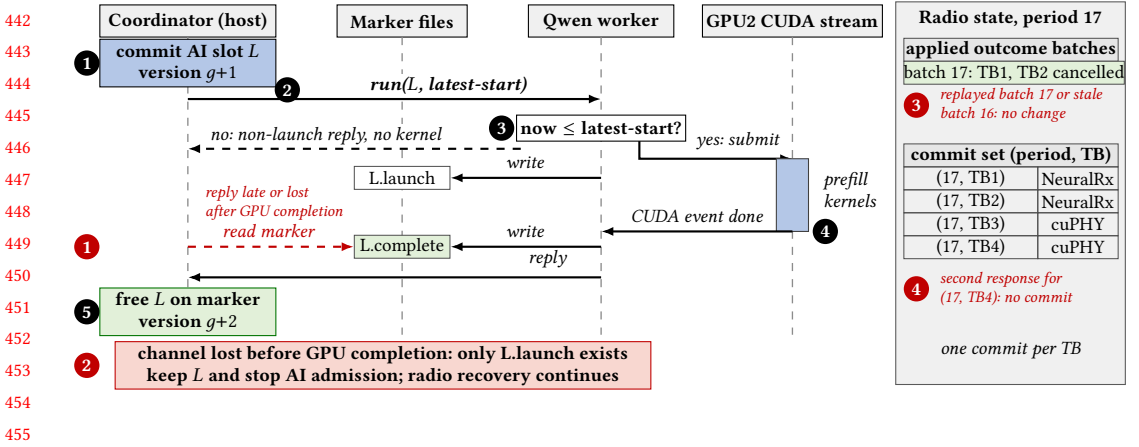


Fig. 7. Enforcing one AI slot on the GPU. Black steps: commit with a latest-start (1), run request (2), launch gate and launch marker (3), CUDA event and completion marker (4), and freeing on a matching marker (5). Red faults: marker reconciliation after a late reply (1), AI admission stop after channel loss (2), and no-op outcome and response replays (3, 4).

Bounded Launch. The coordinator commits L as version $g + 1$ (1) and sends $run(L, latest-start)$ to the Qwen worker (2). Prepare creates one launch ticket that carries the calendar version, worker epoch, ring and transport credit, and absolute latest-start. The worker launches only when the current time is at or before the latest-start (3); it then writes the launch marker $L.launch$ and submits the prefill kernels to the GPU2 stream. Otherwise, it replies with a non-launch marker and does not execute the stale schedule.

Physical Retirement. When the CUDA event completes, the worker writes the completion marker $L.complete$ and replies (4). The coordinator frees L only on a marker that matches its version and advances to version $g + 2$ (5). RPC completion is not GPU completion. Therefore, slots and credits remain owned until a matching CUDA completion event, and a transport-copy completion cannot replace a receiver or recovery completion.

Fault Handling. The red steps show the injected faults. When the reply is delayed or lost after GPU completion, the coordinator reads $L.complete$ and frees L from the marker (1). When the channel is lost before GPU completion, only $L.launch$ exists at the end of the slot; the coordinator keeps L owned, stops AI admission, and continues radio recovery (2). A replay of the nrx-17 outcome batch or of a period-16 batch leaves the radio state unchanged (3), and a second response for TB4 of period 17 produces no commit because the commit set holds one entry per TB (4). A timeout without physical evidence invalidates the version and closes new optional admission.

Distributed and Transport Ownership. One global AI queue can serve several cuPHY instances, but each instance holds at most one pending launch ticket in an ownership epoch, and commit transfers launch authority only while the local and global schedules remain current. CUDA IPC and NVLink P2P avoid a host-staging copy and also make ownership explicit: each ring entry has one version, one physical completion state, and one scheduler-visible completion event. Therefore, Backstop charges transport inside the qualified service path. Same-worker reconnect carries the version through bounded markers, while full process replacement without durable reconciliation remains UNQUALIFIED.

3.4 Event-scheduling algorithm

Backstop is an event-driven scheduling algorithm with a verified commit point. Algorithm 1 runs at every event batch and rejects optional work whenever measurement conditions, recovery feasibility, or ownership cannot be proven.

Algorithm 1: SCHEDULEEVENT($X_t, \mathcal{E}_t, \mathcal{A}_t, \pi$)

- (1) If any required bound disagrees with π , return UNQUALIFIED and disable new optional work.
- (2) Apply the event batch $\mathcal{E}_t$ atomically: add a pending recovery for each new radio job; cancel it only for a timely success; keep it for failure, lateness, or missing evidence.
- (3) Place every required recovery on its qualified physical lane. If no all-fail calendar exists, return INFEASIBLE and reject new NeuralRx and AI admission.
- (4) Visit only visible AI jobs in deterministic policy order. For each job, construct X' by jointly retiming recovery and adding one complete AI slot.
- (5) If the independent verifier accepts X' , atomically commit it, dispatch its first action, and return SAFE+AI. Otherwise discard X' without changing X_t .
- (6) If no AI candidate passes, commit the recovery-only calendar and return SAFE-ONLY. A matching terminal event ends ownership and invokes SCHEDULEEVENT again.

Fig. 8. Backstop's online scheduling algorithm. Candidate order controls efficiency; the verifier controls safety. Consequently, a heuristic can reject a safe candidate but cannot authorize an unsafe state.

Event Processing. Step 1 disables new optional work and returns UNQUALIFIED when a required bound disagrees with the active measurement conditions π . Step 2 applies the event batch atomically: it adds a pending recovery for each new radio job, cancels a pending recovery only for a timely success, and keeps it for failure, lateness, or missing evidence. Step 3 places every required recovery on its qualified lane and returns INFEASIBLE when no all-fail calendar exists. Steps 4 and 5 visit the visible AI jobs in a deterministic policy order, build one candidate X' per job, and commit the first candidate that the verifier accepts, which returns SAFE+AI. Step 6 commits the recovery-only calendar and returns SAFE-ONLY when no AI candidate passes.

Decision Window. For m_h equal-length recoveries compacted at expiry d_h with margin g_h , let b_{eff} include synchronous control, AI service, and the completion margin. AI-first execution is safe under the sufficient, mode-specific condition

$$T_{\text{dec},h} + b_{\text{eff}} \leq d_h - g_h - m_h c_h. \quad (6)$$

The right side is the latest time at which the first required recovery can start. Therefore, the inequality charges decision and launch delay and does not assume immediate dispatch.

Idle Time. The same window shows why idle GPU time cannot authorize admission.

PROPOSITION 2 (IDLE TIME IS INSUFFICIENT). *On one non-preemptive lane with common recovery deadline D , unresolved bounds $c_1, \dots, c_m$, and AI bound b_{eff} , suppose mandatory-only execution is feasible but*

$$t + b_{\text{eff}} + \sum_{i=1}^m c_i > D. \quad (7)$$

Then admitting AI into idle time while ignoring pending recovery cannot guarantee the radio contract.

PROOF. Choose the allowed realization in which AI and every recovery take their declared bounds. The final recovery completes after D by Equation 7; present idle time does not erase future mandatory demand. $\square$

Table 3. The scheduler's predictive outcomes.

Class	Meaning
SAFE+AI	All-fail recovery is executable and at least one qualified AI request is admissible.
SAFE-ONLY	All-fail recovery is executable, but the visible AI candidate cannot safely fit.
INFEASIBLE	The mandatory all-fail schedule is infeasible under known bounds.
UNQUALIFIED	A required bound was not measured under the active conditions or failed qualification.

Safe Reclaimable Capacity. The same invariant predicts the capacity that the algorithm can expose. Let W be the window up to the recovery deadline, L the AI deadline, a one complete AI transaction, c one recovery bound, M_g the pending recoveries that static reservation holds, and F_g the required recoveries on lane g . With $[x]_+ = \max(0, x)$,

$$K_{\text{static}} = \sum_g \left\lfloor \frac{[\min(L, W - M_g c)]_+}{a} \right\rfloor, \quad (8)$$

$$K_{\text{RF}} = \sum_g \left\lfloor \frac{[\min(L, W) - F_g c]_+}{a} \right\rfloor, \quad (9)$$

$$K_{\text{Backstop}} = \sum_g \left\lfloor \frac{[\min(L, W - F_g c)]_+}{a} \right\rfloor. \quad (10)$$

Static reservation holds every pending recovery, recovery-first executes the required recoveries before AI, and Backstop may place AI first while keeping $F_g c$ before the recovery deadline. Multi-class BurstGPT batches use exact subset, lane, order, and recovery-insertion enumeration. Failure probabilities predict expected value, while the all-fail safety check does not depend on the failure distribution.

Predictive Outcomes. Table 3 lists the four outcomes of a decision. They keep scheduling failure distinct from missing authority: SAFE+AI and SAFE-ONLY preserve the same radio invariant, INFEASIBLE proves overload under known bounds, and UNQUALIFIED refuses to reuse evidence from a different physical mode.

3.5 Guarantee scope

Backstop provides a conditional guarantee for an explicitly qualified mode. Qualification binds service bounds to the environment in which they were measured and defines the events that the runtime must contain.

Mode Conditions. A mode fixes GPU topology, MPS settings, process lifecycle, worker epoch, memory residency, co-run class, software, and finite-sample whole-path bounds. Changing any element requires requalification. The synthetic release period and result expiry used in evaluation are harness parameters, not NR slot or production HARQ timing.

Contained Events. The runtime handles NeuralRx failure and lateness, stale and duplicate messages, bounded reply loss before or after a control-plane effect, missing AI replies after physical completion, and same-worker channel reconnection. It stops new AI admission when completion evidence is absent.

Excluded Events. Arbitrary GPU hangs, driver reset, unbounded host suspension, production-clock uncertainty, and unqualified workloads remain outside the claim. Mandatory offered load without an all-fail schedule is INFEASIBLE rather than a reason to weaken the safety check.

3.6 Backstop implementation

Backstop binds the model to persistent CUDA workers and explicit control objects, which makes schedule state observable and enforceable across process and GPU boundaries.

Radio and AI Workers. MIG is disabled and MPS is enabled. cuPHY instances, NeuralRx endpoints, conventional recovery, and external AI run in separate persistent workers. TensorRT NeuralRx workers hold preallocated bindings and CUDA graphs, persistent Aerial cuPHY workers execute conventional recovery, and a Qwen2.5-1.5B worker executes bounded prefill classes. In the evaluated placement (Figure 5b), GPU0 and GPU1 host the cuPHY instances with a 50% MPS thread limit, GPU3 hosts the NeuralRx endpoint, and GPU2 hosts the shared cuPHY lane and the Qwen worker, which runs as a low-priority MPS client with a 20% thread limit.

Transport and Control. CUDA IPC rings connect processes on one GPU, and NVLink P2P rings connect GPUs. Each slot carries a request version and a physical completion state. Copy completion does not imply receiver or recovery completion; the owning worker publishes a separate completion event. The controller stores recovery calendars, endpoint and transport credits, worker epochs, global AI ownership, and single-commit state. Unix-domain RPC carries control messages, while CUDA-resident data stays on the IPC and P2P paths.

Schedule Builder and Verifier. Backstop separates schedule construction from admission authority. The candidate scheduler applies deterministic list orders to radio jobs and visible AI candidates and emits a complete schedule, an ownership delta, and a mode fingerprint; exact search serves only as a small-state oracle. The independent verifier checks job identity, release, qualified service, expiry, lane capacity, overlap, AI busy intervals, credit uniqueness, and version, and the runtime cannot execute a candidate that the verifier rejects. The executor performs the first non-preemptive action of an accepted schedule, observes its terminal event, and replans; it never reorders physical work from an unverified host-side queue.

Measurement and Mode Separation. Instrumentation separates logical decisions, host progress, GPU execution, and physical completion, which prevents a host timestamp from being reported as a kernel boundary. The runtime records release, admission, dispatch, worker observation, CUDA completion, commit return, request and payload identity, ring version, worker epoch, and slot freeing time, and Nsight traces check GPU ordering for selected modes. The qualified C158-C162 path uses pyAerial with persistent workers. A later native C++ effort verifies input-layout and decoder parity only; it is not timing-qualified, and its measurements are never mixed with the service bounds of the Python mode. Manifests preserve source hashes, protocols, seeds, node exclusions, analyzer versions, failures, and result hashes, and every empirical block in this paper maps to a machine-readable artifact and field set.

4 Evaluation

4.1 Evaluation setup

Hardware Specification. We evaluate Backstop on two independent nodes, each with four NVIDIA A100-SXM4-40GB GPUs connected by third-version NVLink. MIG is disabled and MPS is enabled on every GPU. In the multi-GPU placement, GPU0 and GPU1 run the two cuPHY instances at a 50% MPS share, GPU3 runs the TensorRT NeuralRx worker at 80%, and GPU2 runs the shared cuPHY recovery lane at 80% and the Qwen worker at 20% with low priority. Facility identity is withheld for anonymous review.

Evaluation Scope. The trace-driven evaluation (Sections 4.2 and 4.3) measures safe AI capacity and timely AI value with an exact-checked capacity model and BurstGPT traces, without executing kernels. The physical evaluation (Sections 4.4-4.8) runs the complete scheduler on both nodes and measures online scheduling, AI service under bursty arrivals, radio safety at the admission

boundary, schedule enforcement under injected faults, prediction accuracy, and decision cost. Section 4.9 varies the recovery bound, the MPS configuration, and the worker lifecycle.

Workloads and Contract. Radio execution uses NVIDIA Aerial cuPHY [14] and a TensorRT FP16 NeuralRx. The evaluated warm mode uses a synthetic 180 ms release period and a 155 ms result expiry (P180/D155), with finite-sample whole-path bounds $B_{\text{NRx}} = 45$ ms, $B_{\text{conv}} = 25$ ms, and $B_{\text{AI}}(\text{context64}) = 35$ ms and a 2 ms margin before the expiry. The period is not an NR slot, and the expiry is not a production HARQ deadline. In every period, four admitted cells each send one single-UE, 273-PRB uplink TB; the cuPHY instance on GPU0 serves three of them and the instance on GPU1 serves one. Qwen2.5-1.5B prefill [17] with 16-512-token prompts creates external GPU contention, and BurstGPT [24] supplies arrivals and token values from four non-overlapping public-trace windows and the prompts of the bursty physical campaign. Synthetic channels drive system qualification; a separate Sionna CDL-D/E mode tests radio value (Section 6).

Baselines. Every baseline makes the same radio decisions and sees the same arrivals, bounds, and workers as Backstop. Two baselines follow the sharing approaches of Section 5. *Static reservation* holds the recovery of every admitted TB until expiry, as reservation-based sharing does, and *idle-time admission* admits AI into GPU time that is idle at the decision and ignores pending recovery, as work-conserving sharing does. The *recovery-first* policy is an ablation of Backstop: it keeps pending-recovery tracking, the all-fail admission check, and the single-commit rule of Backstop but runs every required recovery before external AI, which isolates the effect of AI-first order. The *offline oracle* sees the realized trace and bounds the attainable value. Idle-time admission and uncontrolled MPS are diagnostic baselines; they expose broken assumptions and are not safe performance comparators.

Experimental Discipline. Each authoritative campaign freezes its source hash, workload, bounds, seeds, node assignment, exclusions, analyzer, and stop rule before opening the outcome. Development and holdout runs use independent seeds and nodes, and failed, negative, and UNQUALIFIED runs stay in the ledger. We report zero observed events as finite-sample results rather than probability or worst-case execution time (WCET) claims.

Reproducibility. The anonymous artifact package contains source, protocols, manifests, analyzers, exact checker, independent verifier, result summaries, and figure builders. Hardware campaigns require proprietary Aerial/cuPHY and TensorRT components; model, checker, trace, and analysis paths do not. Node names and absolute paths are removed from the review artifact.

Use of generative AI. Generative-AI tools assisted with language editing, organization, bibliography normalization, and consistency-checking scripts. The authors designed the system and model, fixed protocols, executed experiments, inspected failures, and verified every claim against hashed artifacts. Generative AI produced no experimental measurement or ground-truth label.

4.2 Safe AI capacity

Capacity across Cells and Deadlines. Figure 9 evaluates the exact-checked capacity model over 21,384 points that vary cells (one TB per cell and period), cuPHY instances, GPUs, NeuralRx success probability and correlation, expiry, recovery bound, AI class, AI deadline, and offered load. Mandatory recovery is infeasible at 7,668 points. The model decides all AI requests at one common time after the outcome batch, and at all 13,716 feasible points Backstop admits the same AI capacity as the offline oracle. Static reservation reaches 85.3% of the oracle with two cells and 59.3% with 16 cells because it holds every pending recovery until expiry. Recovery-first reaches 92.4% and 83.3% because it runs required recovery before AI; at a 50 ms AI deadline, it reaches only 62.7%. Backstop exceeds static reservation at 3,600 points (26.2%), and recovery-first trails its oracle by at least 5% at 2,604 points (19.0%). Idle-time admission admits more AI than the oracle, but its radio violation

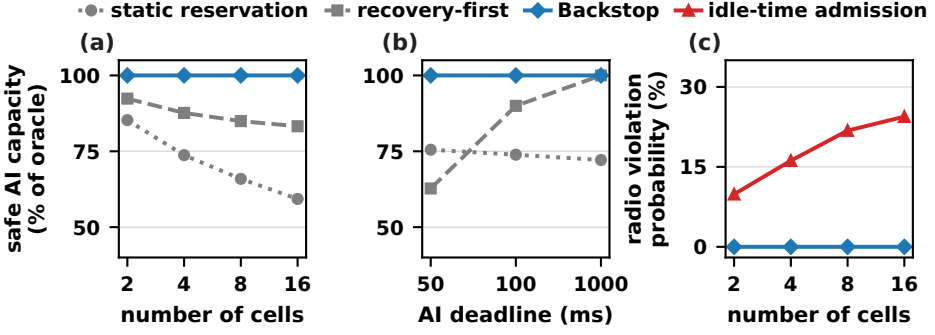


Fig. 9. Safe AI capacity in the exact-checked capacity model (13,716 mandatory-feasible points). (a) AI capacity relative to the offline oracle as the number of cells grows. (b) AI capacity as the AI deadline tightens. (c) Radio violation probability of idle-time admission; static reservation, recovery-first, and Backstop never violate the all-fail schedule. Idle-time admission is omitted from (a) and (b) because the AI that it admits exceeds the safe capacity.

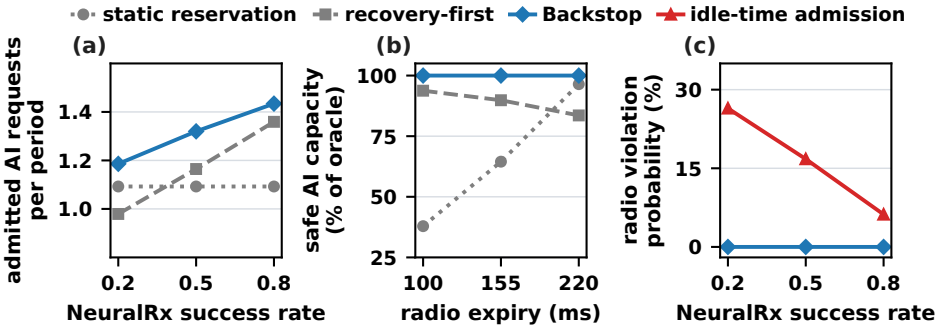


Fig. 10. Drivers of safe AI capacity in the exact-checked capacity model. (a) Mean AI requests per release period against the NeuralRx success rate. (b) AI capacity relative to the offline oracle against the radio expiry. (c) Radio violation probability of idle-time admission against the NeuralRx success rate. Each success rate covers the same 4,572 mandatory-feasible configurations.

probability rises from 9.9% with two cells to 24.4% with 16 cells and is positive at 4,338 feasible points (31.6%). Exhaustive order checking agrees with the compact capacity calculation in all 5,376 checked cases. Backstop achieves this capacity by admitting AI against the all-fail schedule of the remaining recoveries and by placing AI first whenever that schedule still completes before the recovery deadline.

Where the Throughput Comes From. Tracking pending recovery is the main source of the throughput gain. Static reservation holds every pending recovery until expiry and reaches only 59.3-85.3% of the AI capacity of Backstop across cell counts. Recovery-first tracks pending recovery like Backstop and reaches 83.3-92.4%, because both policies fit the same AI work and required recoveries into one window. The remaining gap comes from order: recovery-first loses the AI requests whose deadline passes while the required recoveries run first. Therefore, at a 1,000 ms AI deadline recovery-first reaches 100% of the capacity of Backstop, and at a 50 ms deadline it reaches 62.7% (Figure 9b).

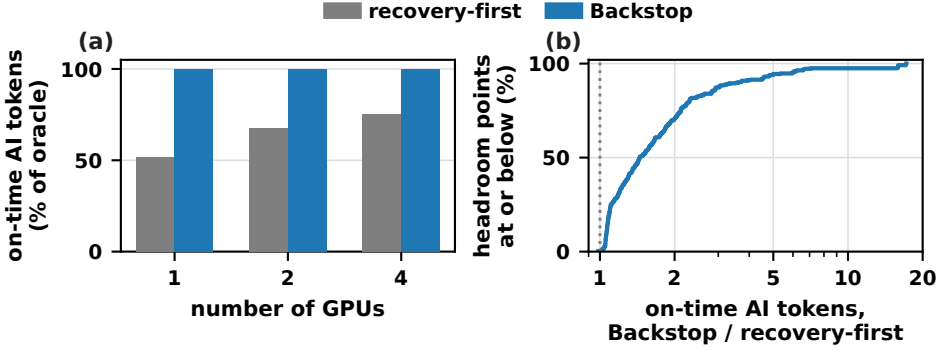


Fig. 11. Timely AI value on BurstGPT traces at the 362 prespecified headroom points; timely value counts the AI tokens served within their SLO. (a) Mean timely value relative to the fixed-placement oracle per GPU count. (b) Cumulative distribution of the ratio between the timely value of Backstop and that of recovery-first.

Capacity Released by NeuralRx Success. Figure 10a isolates the capacity that a timely NeuralRx success releases. Each success rate covers the same 4,572 mandatory-feasible configurations. As the success rate grows from 0.2 to 0.8, Backstop admits 1.19, 1.32, and 1.43 AI requests per period on average, while static reservation stays at 1.09 requests because it holds every pending recovery regardless of the outcome. Recovery-first rises from 0.98 to 1.36 requests and admits fewer requests than static reservation at a 0.2 success rate, because it runs the many required recoveries before any AI request. Backstop converts each success into AI capacity because the outcome batch cancels the pending recovery of every successful TB and Backstop places AI first against the remaining all-fail schedule.

Radio Expiry. Figure 10b varies the radio expiry. At a 100 ms expiry, static reservation reaches only 37.9% of the oracle because the reserved recoveries fill most of the short window; at 220 ms, it reaches 96.5%. Recovery-first follows the opposite trend, from 93.8% to 83.6%, because a longer window makes the AI deadline the binding constraint, and running the recoveries first then delays AI past its deadline. Backstop stays at the oracle capacity at every expiry. Figure 10c shows the cost of ignoring pending recovery: the violation probability of idle-time admission falls from 26.4% to 6.2% as the success rate grows, and it is largest when NeuralRx fails most often.

4.3 Timely AI value on real traces

Trace Headroom. A frozen screen maps four BurstGPT windows onto 2,592 topology, failure, recovery, AI-SLO, and offered-load points, of which 2,052 are mandatory-feasible. At a 50 ms AI SLO, even the oracle places no AI request. At 100, 250, and 1,000 ms, recovery-first reaches 75.8%, 98.0%, and 100% of the oracle on average. The prespecified screen advances the 362 points at which recovery-first trails the oracle by at least 5% in every window: 298 at a 100 ms SLO and 64 at 250 ms.

Value at the Headroom Points. Figure 11a shows that Backstop closes this gap. On one GPU, recovery-first delivers 51.7% of the fixed-placement oracle and Backstop delivers 99.8%; on two and four GPUs, recovery-first delivers 67.4% and 75.5%, while Backstop stays above 99.8%. Figure 11b shows that Backstop delivers 1.44× the timely value of recovery-first at the median and up to 17.2×. The AI-first greedy order of Backstop is within 5% of the fixed-placement oracle at 361 of the 362 points, and the remaining point has a 7.02% gap. Backstop achieves this by placing AI before required recovery whenever the all-fail schedule still ends before the recovery deadline, while recovery-first delays every AI request behind the required recoveries and misses tight AI

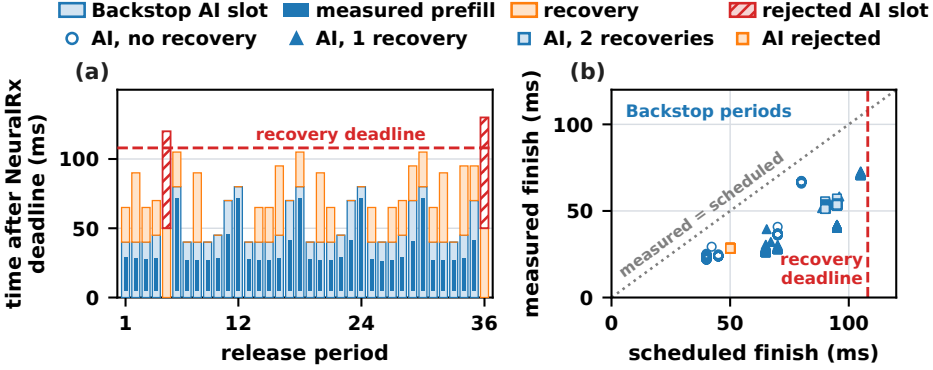


Fig. 12. Online scheduling in the two-node campaign, measured from the NeuralRx deadline. (a) Committed schedule of the shared lane in 36 consecutive release periods: each admitted AI slot includes the 5 ms launch budget, and the required recoveries follow it with their full bound; a rejected AI slot is drawn where it would end. (b) Measured against scheduled finish of the last action on the shared lane, the AI prefill or the last recovery, in all 1,200 periods; the marker gives the required recoveries of the period, orange marks a rejected AI request, and the dotted diagonal marks equality.

SLOs. Flexible recovery placement across GPUs has a positive upper-bound gain at 203 points: 0/102 one-GPU, 145/175 two-GPU, and 58/85 four-GPU points, with a 10.64% median positive gain. Because this oracle excludes movement cost and physical qualification, it is a design target, and we make no executed optimizer claim. Selecting the frozen Sionna strata to minimize pending recoveries under zero or 0.02 absolute radio-value loss removes no pending recovery.

Original Physical Mode. The advantage depends on the headroom that the workload leaves. The calibration-only exact screen of the original physical mode gives static reservation, recovery-first, Backstop, and the oracle the same radio decisions, arrivals, bounds, and one-Qwen-per-period capacity. After correcting a preserved absolute-deadline interpretation defect without excluding a request or opening the holdout, exact matching assigns 856 timely requests and 349,387 tokens to recovery-first, Backstop, and the oracle, and static reservation exposes no qualified same-period Qwen slot. Charging every required recovery its full bound leaves recovery-first at 345,405 tokens, only 1.14% below Backstop and below the prespecified 5% gate. Thus, the original mode contains no ordering headroom, and recovery-first remains the strong safe baseline in that mode.

4.4 Online scheduling on physical GPUs

Two-Node Campaign. The frozen P180/D155 campaign runs 600 release periods on each of two nodes and offers one Qwen request of 16-512 tokens in every period. It executes 4,800 TensorRT NeuralRx requests, 1,312 shared-cuPHY recoveries, 1,077 Qwen requests, and 4,800 single radio commits without a violation of any deadline, bound, credit, or commit rule; 0/4,800 deadline misses gives a 95% rule-of-three upper bound of 0.0625%. Maximum NeuralRx, recovery, and radio-commit paths are 18.123, 13.465, and 118.022 ms against contracts of 45, 25, and 155 ms. Nsight verifies 191 NeuralRx-Qwen kernel-overlap segments totaling 8.932 ms and conventional overlap with the NeuralRx of another cell in 12/12 windows for 12.309 ms, which establishes physical co-execution rather than service bounds.

AI-First Schedules. Figure 12a shows the schedules that Backstop commits in 36 consecutive periods of the development run. In every admitted period, the AI slot takes the shared lane first, and the required recoveries follow it with their full 25 ms bound and still end before the recovery

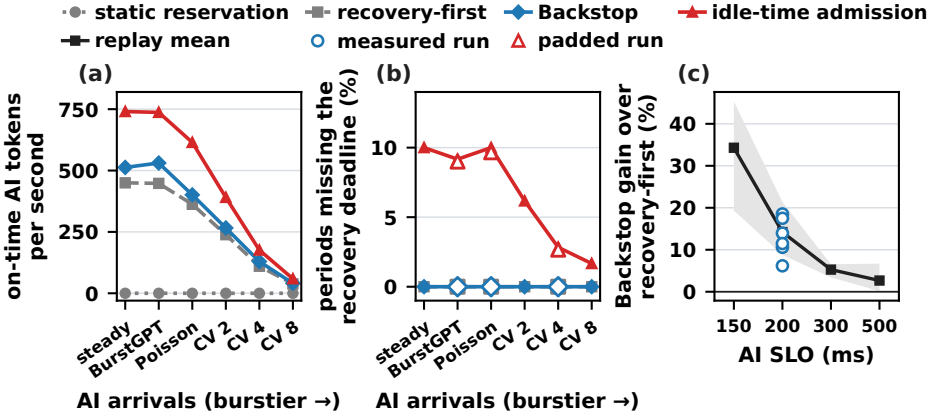


Fig. 13. AI service and radio safety under bursty AI arrivals on the physical two-node path. Every pattern carries the same 329 BurstGPT prompts at 3 requests/s for 108 s and differs only in arrival times; the AI SLO is 200 ms. (a) AI tokens served within their SLO; static reservation admits no AI request. (b) Periods in which the admitted AI misses the recovery deadline at the declared bounds (lines, measured service times), and periods whose last recovery physically ends after the recovery deadline when every AI request and recovery takes its bound (padded runs). (c) Gain of Backstop over recovery-first against the AI SLO: replay mean over the six patterns with its range (band), and the measured runs at 200 ms.

deadline, 108 ms after the NeuralRx deadline. Backstop rejects an AI request only when its slot and the required recoveries would cross the recovery deadline, as for the 256-token request of period 5 and the 512-token request of period 36, which each meet two required recoveries. Static reservation holds all four pending recoveries, or 100 ms of this window, and admits no AI request in this mode. **Execution inside the Schedule.** Figure 12b compares, for every period, the measured end of the last lane action with the end that its schedule allows. Every period ends before its scheduled end, and every scheduled end lies before the recovery deadline. The latest measured end is 72.9 ms after the NeuralRx deadline, 35.1 ms before the recovery deadline, and the smallest margin to a scheduled end is 12.7 ms. The margin exists because the schedule charges every AI request its class bound and every recovery its full bound, while Backstop frees a slot at its CUDA completion and replans the required recoveries from that time. In two periods, the coordinator applies the outcome batch 23.8 and 31.0 ms after the NeuralRx deadline; Backstop verifies the slot from the actual decision time, and both periods end before their scheduled ends. Thus, the committed schedule is executable on the physical lane, and the schedule charges the decision delay that it observes.

4.5 AI service under bursty arrivals

Bursty Workload. AI requests arrive in bursts, and during a burst the scheduler decides how much AI each recovery window can absorb. Therefore, we replay one sequence of 329 BurstGPT prompts on the physical two-node path at 3 requests/s for 108 s. The prompts are drawn from all BurstGPT requests and rounded up to the six prompt lengths, and 47.4% of them round to 512 tokens. Six arrival patterns share this sequence and differ only in arrival times: evenly spaced arrivals, the per-second arrivals of a BurstGPT segment, and Gamma arrivals with an inter-arrival CV of 1 (Poisson), 2, 4, and 8. Every request has a 200 ms SLO for its first token, and up to four AI slots share one period in FIFO order. All policies of a pattern observe the same NeuralRx outcome sequence, which makes each comparison paired.

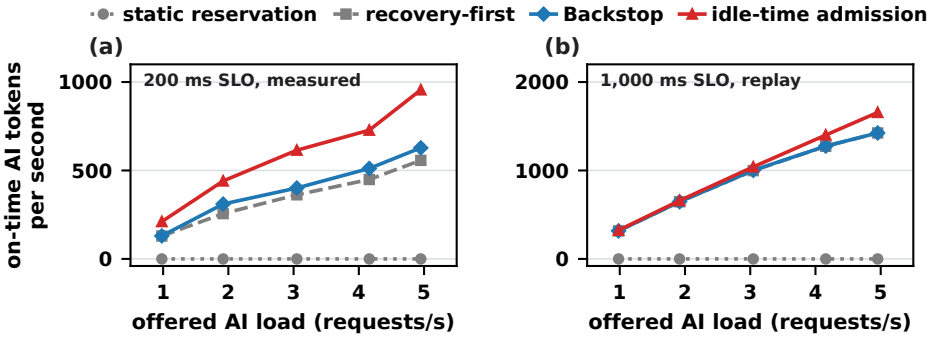


Fig. 14. On-time AI throughput under increasing load with Poisson arrivals. (a) Measured runs with a 200 ms SLO; the 3.05 requests/s point is the Poisson run of Figure 13, taken on the other node. (b) Replay with the measured service distributions and a 1,000 ms SLO. Static reservation admits no AI request in this mode.

AI Service. Figure 13a shows the AI tokens served within their SLO. Backstop serves 6.2-18.5% more on-time tokens than recovery-first under every pattern: 18.5% for BurstGPT arrivals, 10.6% for Poisson arrivals, and 6.2% for CV-8 bursts. Static reservation holds all four recoveries and serves no AI request in this mode. Idle-time admission serves $1.36\text{--}1.53\times$ the tokens of Backstop because it also admits requests whose recoveries no longer fit before the recovery deadline. As the arrivals become burstier, every policy serves fewer tokens because a burst exceeds the window of one period and its requests expire.

Radio Safety under Bursts. Figure 13b shows the price of that extra AI. Idle-time admission misses the recovery deadline at the declared bounds in 1.7-10.0% of periods, most often for smooth arrivals, where a request waits in almost every period. When every AI request and recovery takes its declared bound, its last recovery physically ends after the recovery deadline in 2.7-9.7% of periods, and 128 radio results miss the expiry. Backstop and recovery-first never miss the recovery deadline, and no radio result misses the expiry in any of their 18 runs. Under the same padded execution, Backstop serves 10.6-18.5% more on-time tokens than recovery-first.

Throughput under Load. Figure 14 raises the offered AI load with Poisson arrivals. With a 200 ms SLO, Backstop serves 10.6-21.2% more on-time AI tokens than recovery-first from 1.92 to 4.95 requests/s, and the two nearly tie at 0.98 requests/s. Static reservation serves no AI request at any load, and idle-time admission serves the most but breaks the recovery deadline at the declared bounds in 19-88 of 600 periods. With a 1,000 ms SLO (Figure 14b), Backstop and recovery-first reach the same throughput at every load because both fit the same AI work and required recoveries into each window. Thus, tracking pending recovery sets how much AI one window can hold, and AI-first order sets how much of it meets a tight deadline.

Where the Gain Comes From. Figure 13c varies the AI SLO in a replay with the measured service distributions of the two-node campaign, averaged over five seeds. The gain of Backstop over recovery-first shrinks as the SLO loosens: 34.3% on average at 150 ms, 14.2% at 200 ms, 5.3% at 300 ms, and 2.7% at 500 ms. The measured gains at 200 ms, 6.2-18.5%, lie close to the replay range of 8.8-21.4%. Backstop achieves this gain because it places a verified AI slot ahead of the required recoveries, while recovery-first must charge every required recovery its full bound before an AI slot can meet its SLO. Thus, the ordering matters for interactive AI SLOs and fades for loose ones, consistent with the tie of the original mode in Section 4.3.

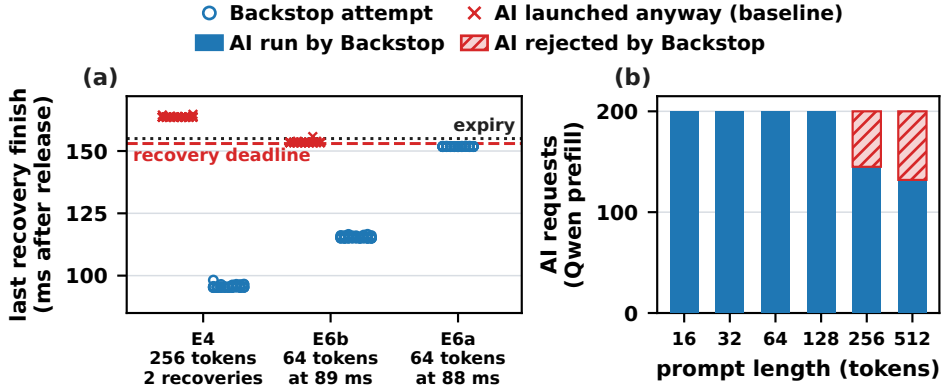


Fig. 15. Radio safety at the admission boundary on two nodes. (a) End of the last recovery in every attempt of the bound-padded boundary diagnostic. E4 offers a 256-token request with two required recoveries at 45 ms; E6b and E6a offer a 64-token request with one required recovery at 89 and 88 ms. Baseline attempts launch the request anyway: idle-time admission in E4 and a forced launch of the request that Backstop rejects in E6b. (b) AI requests per prompt length in the two-node campaign: requests that Backstop runs and requests that it rejects because the required recoveries would then miss the recovery deadline; idle-time admission would run all of them, and static reservation runs none.

4.6 Radio safety at the admission boundary

What the Admission Check Prevents. Figure 15a runs three prespecified boundary cases on two independent nodes as a bound-padded diagnostic, in which every recovery and AI request takes its declared bound. In E4, two required recoveries remain at 45 ms and a 256-token AI request is offered; idle-time admission launches it, and the last recovery ends 10.6-11.7 ms after the 153 ms recovery deadline in every attempt. In E6b, one recovery remains and a 64-token request arrives at 89 ms; launching the request that Backstop rejects moves the last recovery 0.4-2.8 ms past the recovery deadline. Backstop rejects both requests, admits the adjacent 88 ms case E6a, and ends every attempt before the recovery deadline. Across the selected rounds, idle-time admission in E4 misses both the recovery deadline and the expiry in 40/40 rounds, the forced launch in E6b misses the recovery deadline in 40/40, and Backstop records 0/120 safety violations (95% rule-of-three upper bound 2.5%). Jointly bound-valid counts are 39/39, 39/39, and 0/119, respectively.

Rejected Requests. Figure 15b shows the same boundary in the two-node campaign. Backstop executes every offered 16-128-token request and rejects 55 of the 256-token and 68 of the 512-token requests before launch. These are exactly the requests that would push the last of two required recoveries past the recovery deadline. Idle-time admission would launch all 1,200 requests, including these 123, and static reservation would launch none. Thus, Backstop admits every request that the all-fail schedule allows and no request that it forbids.

4.7 Schedule enforcement and fault containment

Latest-Start Enforcement. A committed schedule remains valid only if the GPU cannot start a slot later than the schedule assumes. Figure 16a shows the arrival of each slot at the Qwen worker relative to its absolute latest-start. All 1,077 slots of the two-node campaign arrive 2.9-3.8 ms before the latest-start and launch. The held-slot fault keeps 52 slots until 6.4-7.2 ms after the latest-start, and the worker refuses all 52 before GPU launch with a non-launch marker. A launch that ignored the latest-start would run these requests up to 7.2 ms later than their schedules allow and push the

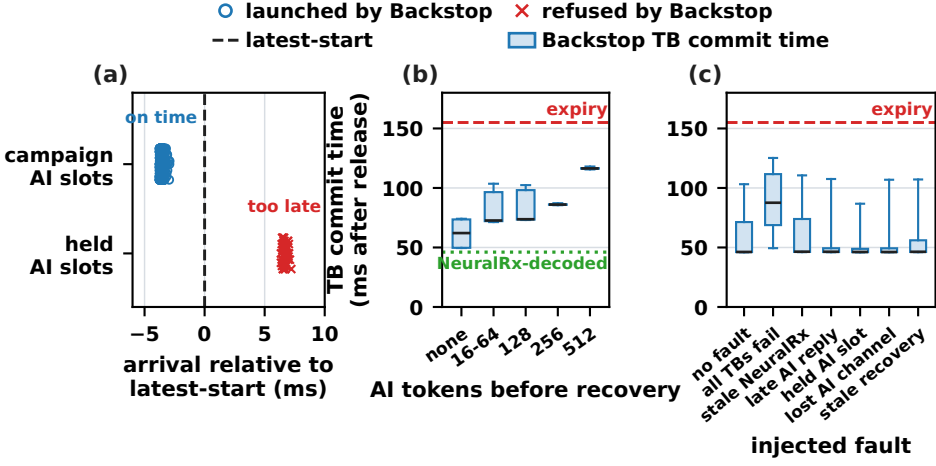


Fig. 16. Schedule enforcement and radio commit time. (a) Arrival of each AI slot at the Qwen worker relative to its absolute latest-start: the 1,077 slots of the two-node campaign and the 52 slots that the held-slot fault keeps past the latest-start. (b) Commit time of every recovered TB in the two-node campaign, grouped by the prompt length of the AI slot that runs before its recovery; the dotted line marks the median commit of the TBs that NeuralRx decodes. (c) Commit time of every TB under each injected fault: no fault, all TBs of a period fail, stale and duplicate NeuralRx outcomes, a late AI reply after GPU completion, an AI slot held past its latest-start, an AI channel lost before GPU completion, and stale and duplicate recovery responses.

required recoveries toward the recovery deadline. Backstop achieves this by binding the absolute latest-start to the slot at commit and by checking it in the worker immediately before kernel submission.

Radio Commit Time. AI-first placement delays the required recoveries, and Figure 16b measures this cost on every TB of the two-node campaign. The 3,488 TBs with a timely NeuralRx success commit in the outcome batch at a median of 46.1 ms after release, independent of the AI slot. Recovered TBs commit at a median of 62.1 ms when no AI slot precedes the recovery, 72.7-73.7 ms after a 16-128-token slot, 86.0 ms after a 256-token slot, and 116.4 ms after a 512-token slot. The latest commit, 118.022 ms, precedes the 155 ms expiry by 37.0 ms. Thus, AI-first placement moves a recovery commit later by at most the scheduled slot length and never past the expiry, because the schedule charges each slot before the recoveries that follow it.

Fault Containment. A stale message or a lost reply can free the wrong physical credit even when the schedule is feasible. Therefore, we combine finite-state checking with physical fault injection. A finite-state audit checks 51 combinations of worker epoch, version, completion, and marker state without an invariant violation. The physical campaign injects seven fault classes on the two qualified nodes. Figure 16c shows that every TB in every class commits exactly once before expiry. The largest commit time is 125.2 ms when every TB of a period fails and all four recoveries run, and it stays 29.8 ms before the 155 ms expiry. The campaign executes 700 release periods, 2,800 NeuralRx requests, 942 recoveries, and 2,800 commits without an observed deadline miss, including 40 replay pairs, four terminal faults, and 260 continuation periods; 0/2,800 misses gives a 95% rule-of-three upper bound of 0.107%. One additional node violates the NeuralRx whole-path bound on its first round, and we retain that run as UNQUALIFIED without pooling it. Thus, fault containment applies to the two qualified nodes and the declared fault and lifecycle classes.

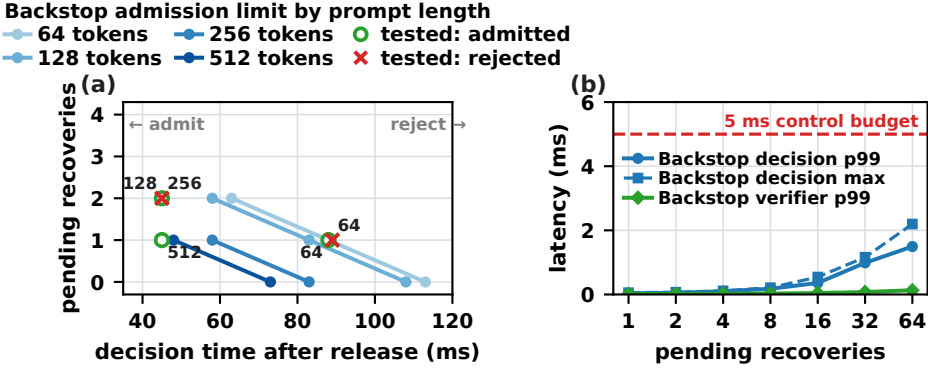


Fig. 17. Predictive scheduling and control cost. (a) Latest decision time at which one AI request of each prompt length remains admissible with four admitted TBs and zero to four pending recoveries; circles and crosses mark the prespecified physical tests that the scheduler admitted and rejected, labeled with their prompt length. (b) Candidate construction and independent verification latency against the number of pending recoveries, with the evaluated 5 ms control budget. Latencies are finite-run measurements, not WCETs.

4.8 Prediction accuracy and decision cost

Prediction Agreement. Figure 17a shows the latest decision time at which each AI class remains admissible as pending recoveries accumulate. The admission limit matches exact feasibility on 16,023 states and all 1,200 retrospective physical decisions. We then freeze seven boundary cases, their order, and node-specific seeds. Across two independent nodes and 180 rounds, model and runtime agree in every round: a 64-token request at 88 ms is admitted in 30/30 rounds, while 89 ms or later is rejected in 30/30. The campaign executes 720 NeuralRx requests, 330 recoveries, 90 Qwen requests, and 720 commits without an observed deadline miss.

Decision Cost. On 600 heterogeneous small states, exact search accepts 504 and the candidate scheduler accepts 494, with zero unsafe acceptances and ten needless rejections; on the qualified mode, candidate and exact acceptance match all 16,023 states. Figure 17b scales the candidate to 64 pending recoveries on a 2,800-state grid with lane capacities of 1, 2, 4, and 8. Every returned schedule passes the verifier. At 64 pending recoveries, the decision latency is 1.494 ms p99 and 2.197 ms maximum, and the verifier latency is 0.131 ms p99, within the 5 ms launch budget. Backstop achieves this cost because the candidate list-schedules only visible jobs and the independent verifier checks one proposed schedule.

4.9 Sensitivity analysis

Recovery Bound. The boundary cases use the qualified $B_{\text{conv}} = 25$ ms contract. Figure 18a shows how the excess of a rejected request depends on this bound. The excess of E4 grows by 2 ms and that of E6b by 1 ms per millisecond of bound because each required recovery adds its bound to the end of the last recovery, and the physical attempts measured at the qualified bound lie within 1.8 ms of these lines. E6b disappears at or below 24 ms, while E4 remains until 19 ms. Figure 18b shows that the 1,312 measured recovery paths have a median of 3.404 ms and a maximum of 13.465 ms, below both thresholds. A sample maximum cannot replace a service contract; lowering the bound defines another mode and requires requalification, and Proposition 2 still yields a later decision time at which ignoring pending recovery is unsafe, for every positive bound. Therefore, the padded campaign is a necessity diagnostic for the declared contract, not a production-rate or WCET claim.

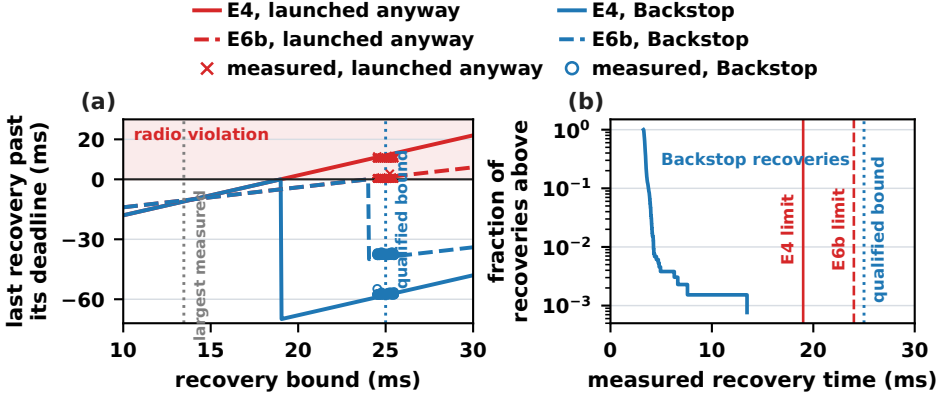


Fig. 18. Sensitivity to the recovery bound. (a) End of the last recovery relative to the recovery deadline when the request that Backstop rejects runs anyway, against the recovery bound B_{conv} : lines follow the bound-respecting schedules of E4 and E6b, and crosses mark the physical attempts measured at the qualified 25 ms bound. (b) Fraction of the 1,312 measured recovery times of the two-node campaign that exceed a given length, with the bounds at which E4 and E6b disappear (E4 and E6b limits) and the qualified bound.

MPS Configuration. MPS controls concurrency but does not bound time. A separate-client overrun produces 2/1,500 deadline misses at a 100% cap and 1/1,500 at a 20% cap, and a matched lifecycle comparison observes 0/10,000 misses for a CPU socket sham and 12/10,000 for the GPU/MPS-client arm. The two campaigns are reported separately and do not imply a monotonic cap effect. Thus, neither the cap selection nor the current queue state authorizes admission.

Worker Lifecycle. Worker lifecycle changes the service distribution on which a schedule relies. The lifecycle matrix contains five qualified or partial subsets and five UNQUALIFIED modes. Qualified evidence covers warm persistent execution, the first request after 30 s idle, MPS restart followed by requalification, radio continuity during Qwen reload, and same-worker reconnect. Optional-service availability during restart or reload is outside the claim. Process and model cold starts, 5 min and 30 min idle, GC-on, and worker process replacement remain UNQUALIFIED. A single-node replacement result is “Development canary only” because it lacks an independent holdout. Durable journal recovery would be a new mechanism and remains outside this paper.

5 Related Work

GPU Sharing in AI-RAN and vRAN. Previous studies on AI-RAN and virtualized RAN (vRAN) platforms focus on sharing compute between deadline-critical RAN functions and other workloads. YinYangRAN multiplexes 5G PHY and ML workloads on GPU accelerators [19], CloudRIC shares pools of heterogeneous accelerators among distributed units [20], and Concordia reclaims vRAN CPU capacity while meeting processing deadlines [7]. Other studies orchestrate AI and RAN workloads through MIG partitions, heterogeneous placement, and worker location [3, 12, 21, 22], while ARCHES switches PHY models, OCUDU exposes in-DU timing classes, and SMEC coordinates RAN and edge SLOs [16, 18, 25].

Our study aligns with these efforts to reclaim RAN compute for other workloads. However, their scheduling units are resources or queued jobs, and their public designs do not represent the recovery that a speculative neural receiver may create. Backstop bridges this gap by tracking each pending recovery and by joining one all-fail recovery calendar across cuPHY instances to an external-AI slot that holds through physical completion.

Real-Time GPU Scheduling and Fault-Tolerant Reservation. Previous studies on GPU scheduling focus on predictable latency for concurrent GPU work through microsecond-scale preemption [9], preemptive scheduling across accelerators [23], spatio-temporal scheduling of real-time DNN inference [4], and hardware compute partitioning [5]. Fault-tolerant real-time scheduling reserves a backup copy for each primary task, deallocates the backup when the primary succeeds, and overloads backups whose primaries cannot fail together [8]. Imprecise computation splits a task into mandatory and optional parts and schedules the optional part only when time remains [13].

While our study aligns with these reservation and preemption techniques, preemption and partitioning control kernels that are already queued, and backup overloading assumes that primaries do not fail together. Backstop bridges this gap by verifying one all-fail schedule for the recovery of every pending receiver, releasing a recovery on a timely success, and enforcing that schedule through GPU launch, transport ownership, and physical completion.

Scoped Difference. Across public designs reviewed through September 2026, we found no system that combines an executable all-fail calendar for unresolved same-TB recovery, external-AI slots held until physical completion, and a predictive admission limit qualified by measurement conditions. This is a scoped literature finding, not a claim about unpublished work or other terminology.

6 Limitations

Claim Boundary. The guarantee is conditional on the qualified bounds and measurement conditions of the evaluated mode. Backstop does not claim that MPS provides hard isolation, that finite-sample maxima are WCETs, or that the synthetic timing contract is a production HARQ deadline. Five lifecycle subsets are qualified or partially qualified and five remain UNQUALIFIED; the claim does not require or imply a 10/10 lifecycle matrix.

Timing Contract. P180/D155 is a synthetic harness contract: P is the inter-release period, not an NR slot duration, and D155 is not a production HARQ deadline. A production d_MAC trace builder and validator pass 24 unit tests, but a target DU/FAPI trace is unavailable. A new vector requires physical requalification; we make no production HARQ guarantee.

Production Timing. Aerial testMAC expects a CRC-bearing UL indication by $T_0 + 4.5$ ms. Under clean, warm, no-AI conditions, local wait-then-recover, same-GPU speculative, and precomputed cross-GPU paths complete 0/1,000 within 4.5 ms. Raw-IQ P2P moves channel estimation through CRC to GPU1 and reaches 852/1,000, and a shared cuPHY/TensorRT stream reaches 885/1,000. The frozen persistent, stream-ordered fast path reaches 995/1,000 with a 3.813 ms median and a 3.995 ms p99, but five late completions reject the gate, and the holdout stays closed. A 300-request profile attributes the remaining tail to remote channel estimation: GPU p50/p99/max are 0.866/4.873/9.592 ms, and its correlation with pair wall time is 0.9603. P2P copies average 36.261/17.150 μ s, derate matching exposes a 1,060.105 μ s host call despite a 0.168 ms GPU p99, TensorRT enqueue averages 7.443 μ s, and the TensorRT graph has a 0.886 ms GPU p99. These profiles are mechanism evidence rather than qualification or WCET. TestMAC is a controlled developer L2 rather than a field-DU d_{MAC} source.

Channel Scope. External Aerial TDL-A remains unsupported. Across raw and normalized inputs, public-reference radio settings, one-antenna frequency/time channel execution, and a public-notebook-style FP32 engine, conventional decoding succeeds in every ten-TB arm and NeuralRx succeeds in none. A fixed development matrix selects Sionna CDL-D/E at 100 ns and retains the A/B/C failures, and a disjoint-seed holdout uses five Es/No strata and 50 blocks per stratum per model. Both pipelines pass 50/50 at 10 dB. CDL-D yields 153/250 conventional and 164/250 neural successes, with 16 neural-only and five conventional-only outcomes, and CDL-E yields 155/250 and 163/250, with 15 neural-only and seven conventional-only outcomes. The aggregate 31 neural-only versus 12 conventional-only outcomes give a two-sided exact $p = 0.00540$: optional NeuralRx

has conditional radio value in the frozen Sionna CDL-D/E mode. That channel has not passed production timing or integrated service-vector qualification.

Finite-Sample Qualification. Zero observed violations establish neither a WCET nor a zero failure probability. Bounds belong to one hardware, software, placement, workload, and lifecycle fingerprint. All authoritative hardware results use A100s, and NVLink P2P does not establish cross-family or inter-node data-plane validity.

Unresolved Long Tail. In C158 attempt 4, one NeuralRx completion reached 350.948 ms while the 400-request run's p99 was 8.111 ms; the run had 4/400 deadline misses. Targeted idle, MPS restart and requalification, Qwen reload, and reconnect campaigns did not reproduce it. Its cause remains unresolved, and GPU and driver hangs remain outside the fault model.

Workload Mapping. BurstGPT supplies real arrivals and token lengths, and Qwen2.5-1.5B prefill is a representative external workload. Token buckets and AI deadlines are experimental mappings; other workloads need new bounds and replay qualification.

7 Conclusion

In this paper, we present Backstop, a recovery-safe multi-GPU scheduler that shares AI-RAN GPUs between neural receivers and AI inference. By tracking each pending recovery and admitting AI only with an executable all-fail schedule, Backstop keeps radio recovery within its deadline while it reclaims the GPU capacity that neural-receiver success releases. Our evaluation shows that Backstop reaches the oracle AI capacity across 13,716 feasible model points and delivers a median of 1.44× the timely AI value of recovery-first at the trace headroom points. On two four-GPU nodes, Backstop records 0/120 safety violations in boundary cases where idle-time admission violates 40/40, serves 6.2-18.5% more on-time AI tokens than recovery-first under bursty AI arrivals, and commits every TB exactly once under seven injected fault classes. These results qualify a synthetic timing mode, not production HARQ timing.

References

- [1] 3GPP. 2020. *Study on Channel Model for Frequencies from 0.5 to 100 GHz*. 3rd Generation Partnership Project. <https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3173> Technical Report 38.901.
- [2] 3GPP. 2024. *NR; Physical Layer Procedures for Data*. 3rd Generation Partnership Project. <https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3216> Technical Specification 38.214.
- [3] Ganesh Ananthanarayanan, Matthew Balkwill, Xenofon Foukas, Zhihua Lai, Bozidar Radunovic, Connor Settle, and Yongguang Zhang. 2025. Distributed AI Platform for the 6G RAN. In *ACM Workshop on Open and AI RAN*. ACM, New York, NY, USA, 15–21. <https://doi.org/10.1145/3737900.3770167>
- [4] Amir Fakhim Babaei and Thidapat Chantem. 2025. DARIS: An Oversubscribed Spatio-Temporal Scheduler for Real-Time DNN Inference on GPUs. In *ACM/IEEE Design Automation Conference*. IEEE, Piscataway, NJ, USA, 1–7. <https://doi.org/10.1109/DAC63849.2025.11132423>
- [5] Joshua Bakita and James H. Anderson. 2025. Hardware Compute Partitioning on NVIDIA GPUs for Composable Systems. In *ECRTS*, Vol. 335. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Wadern, Germany, 21:1–21:25. <https://doi.org/10.4230/LIPIcs.ECRTS.2025.21>
- [6] Sebastian Cammerer, Fayçal Ait Aoudia, Jakob Hoydis, Andreas Oeldemann, Andreas Roessler, Timo Mayer, and Alexander Keller. 2023. A Neural Receiver for 5G NR Multi-User MIMO. In *IEEE Globecom Workshops*. IEEE, Piscataway, NJ, USA, 329–334.
- [7] Xenofon Foukas and Bozidar Radunovic. 2021. Concordia: Teaching the 5G vRAN to Share Compute. In *ACM SIGCOMM*. ACM, New York, NY, USA, 580–596. <https://doi.org/10.1145/3452296.3472894>
- [8] S. Ghosh, R. Melhem, and D. Mossé. 1997. Fault-Tolerance Through Scheduling of Aperiodic Tasks in Hard Real-Time Multiprocessor Systems. *IEEE Transactions on Parallel and Distributed Systems* 8, 3 (1997), 272–283. <https://doi.org/10.1109/71.584093>
- [9] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. 2022. Microsecond-Scale Preemption for Concurrent GPU-Accelerated DNN Inferences. In *USENIX OSDI*. USENIX Association, Boston, MA, USA, 539–558. <https://www.usenix.org/conference/osdi22/presentation/han>

- [10] Mikko Honkala, Dani Korpi, and Janne M. J. Huttunen. 2021. DeepRx: Fully Convolutional Deep Learning Receiver. *IEEE Transactions on Wireless Communications* 20, 6 (2021), 3925–3940. <https://doi.org/10.1109/TWC.2021.3054520>
- [11] Jakob Hoydis, Sebastian Cammerer, Fayçal Ait Aoudia, Avinash Vem, Nikolaus Binder, Guillermo Marcus, and Alexander Keller. 2022. Sionna: An Open-Source Library for Next-Generation Physical Layer Research. arXiv:2203.11854 <https://arxiv.org/abs/2203.11854>
- [12] Haiyuan Li, Yulei Wu, and Dimitra Simeonidou. 2026. Deadline-Driven Hierarchical Agentic Resource Sharing for AI Services and RAN Functions in AI-RAN. arXiv:2605.07547 [cs.NI] <https://arxiv.org/abs/2605.07547>
- [13] Jane W.-S. Liu, Kwei-Jay Lin, W. K. Shih, Albert Chuang shi Yu, Jen-Yao Chung, and Wei Zhao. 1991. Algorithms for Scheduling Imprecise Computations. *Computer* 24, 5 (1991), 58–68. <https://doi.org/10.1109/2.76287>
- [14] NVIDIA Corporation. 2026. *Aerial CUDA-Accelerated RAN Documentation*. NVIDIA. <https://docs.nvidia.com/aerial/cuda-accelerated-ran/>
- [15] NVIDIA Corporation. 2026. *Multi-Process Service Documentation*. NVIDIA. <https://docs.nvidia.com/deploy/mps/>
- [16] Timothy O'Shea, Matthew Pennybacker, and Andriy Kharchenko. 2026. The OCUDU dApp Platform: An Open Runtime and E3 Interface for Real-Time AI-RAN. arXiv:2609.07843 [cs.NI] <https://arxiv.org/abs/2609.07843>
- [17] Qwen Team. 2024. Qwen2.5 Technical Report. arXiv:2412.15115 <https://arxiv.org/abs/2412.15115>
- [18] Neagin Neasamoni Santhi, Davide Villa, Michele Polese, Salvatore D'Oro, Yunseong Lee, Koichiro Furueda, and Tommaso Melodia. 2026. ARCHES: Adaptive Real-Time Switching of AI Models for the RAN. arXiv:2604.23397 [cs.NI] <https://arxiv.org/abs/2604.23397>
- [19] Leonardo Lo Schiavo, Jose A. Ayala-Romero, Andres Garcia-Saavedra, Marco Fiore, and Xavier Costa-Perez. 2024. YinYangRAN: Resource Multiplexing in GPU-Accelerated Virtualized RANs. In *IEEE INFOCOM*. IEEE, Piscataway, NJ, USA, 721–730. <https://doi.org/10.1109/INFOCOM52122.2024.10621380>
- [20] Leonardo Lo Schiavo, Gines Garcia-Aviles, Andres Garcia-Saavedra, Marco Gramaglia, Marco Fiore, Albert Banchs, and Xavier Costa-Perez. 2024. CloudRIC: Open Radio Access Network (O-RAN) Virtualization with Shared Heterogeneous Computing. In *ACM MobiCom*. ACM, New York, NY, USA, 558–572. <https://doi.org/10.1145/3636534.3649381>
- [21] Syed Danial Ali Shah, Maryam Hafeez, Abdelaziz Salama, and Syed Ali Raza Zaidi. 2025. Proactive AI-and-RAN Workload Orchestration in O-RAN Architectures for 6G Networks. arXiv:2507.09124 [cs.NI] <https://arxiv.org/abs/2507.09124>
- [22] Syed Danial Ali Shah, Zeinab Nezami, Maryam Hafeez, and Syed Ali Raza Zaidi. 2025. The Interplay of AI-and-RAN: Dynamic Resource Allocation for Converged 6G Platform. In *IEEE INFOCOM Workshops*. IEEE, Piscataway, NJ, USA, 1–6. <https://doi.org/10.1109/INFOCOMWKSHPS65812.2025.11152939>
- [23] Weihang Shen, Mingcong Han, Jialong Liu, Rong Chen, and Haibo Chen. 2025. XSched: Preemptive Scheduling for Diverse XPU's. In *USENIX OSDI*. USENIX Association, Boston, MA, USA, 671–692. <https://www.usenix.org/conference/osdi25/presentation/shen-weihang>
- [24] Yuxin Wang, Yuhang Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yezu Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. 2025. BurstGPT: A Real-World Workload Dataset to Optimize LLM Serving Systems. In *ACM SIGKDD*. ACM, New York, NY, USA, 5831–5841. <https://doi.org/10.1145/3711896.3737413>
- [25] Xiao Zhang and Daehyeok Kim. 2026. Enabling SLO-Aware 5G Multi-Access Edge Computing with SMEC. In *USENIX NSDI*. USENIX Association, Boston, MA, USA, 1759–1776. <https://www.usenix.org/conference/nsdi26/presentation/zhang-xiao>